

Group Assignment
Name: Santo Giriso David
Roll Number: 202512026
presentation
Computer programming c++

we are presenting **simple, highly preferred, and classic** C++ project that always we go with a **Bank Account Management System**.

this topic because it is incredibly easy to understand, yet perfectly showcases how **Object-Oriented Programming (OOP)** works in the real world.

Here is a simplified blueprint for our project, including slide notes and the code.

The Presentation Outline

Slide 1: Title

- **Title:** Bank Account Management System
- **Subtitle:** A Simple Console-Based Banking Application in C++
- **Talk about:** "I chose this project because banking logic perfectly mirrors Object-Oriented Programming. Features like depositing, withdrawing, and checking balances represent real-world actions handled cleanly by C++ code."

Slide 2: Core Features

- **Create Account:** Initialize an account with a name, account number, and starting balance.
- **Deposit Money:** Add funds safely to the balance.
- **Withdraw Money:** Remove funds, with a safety check to ensure the user doesn't withdraw more than they have (prevents negative balances).
- **Check Balance:** Instantly print the account holder's details.

Slide 3: OOP Concepts Used

- **Encapsulation:** We bundle the account data (balance, name) and the functions (deposit, withdraw) together inside a single BankAccount class. This keeps data safe from accidental corruption outside the class.

The Simplified C++ Code

This code is written to be clean, short, and very easy to explain during your presentation.

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// The Core Class: Encapsulates all bank data and actions
```

```

class BankAccount {
private:
    string accountHolder;
    int accountNumber;
    double balance;

public:
    // Constructor to initialize the account
    BankAccount(string name, int accNum, double initialBalance) {
        accountHolder = name;
        accountNumber = accNum;
        balance = initialBalance;
    }

    // Function to add money
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            cout << "Successfully deposited $" << amount << "\n";
        } else {
            cout << "Invalid deposit amount!\n";
        }
    }

    // Function to withdraw money with a safety check
    void withdraw(double amount) {
        if (amount > balance) {
            cout << "Error: Insufficient funds! You only have $" << balance << "\n";
        } else if (amount <= 0) {
            cout << "Invalid withdrawal amount!\n";
        } else {
            balance -= amount;
            cout << "Successfully withdrew $" << amount << "\n";
        }
    }

    // Function to show account details
    void displayBalance() {
        cout << "\n--- Account Summary ---" << "\n";
        cout << "Holder: " << accountHolder << "\n";
        cout << "Account Number: " << accountNumber << "\n";
        cout << "Current Balance: $" << balance << "\n";
        cout << "-----\n";
    }
};

int main() {
    cout << "=== Creating New Bank Account ===\n";

```

```

string name;
int accNum;
double initialDeposit;

cout << "Enter Account Holder Name: ";
getline(cin, name);
cout << "Enter Desired Account Number: ";
cin >> accNum;
cout << "Enter Initial Deposit: $";
cin >> initialDeposit;

// Creating the object
BankAccount myAccount(name, accNum, initialDeposit);

int choice;
double amount;

// Simple Interactive Menu
do {
    cout << "\n1. Deposit Money\n";
    cout << "2. Withdraw Money\n";
    cout << "3. Check Balance & Details\n";
    cout << "4. Exit Application\n";
    cout << "Enter your choice (1-4): ";
    cin >> choice;

    switch (choice) {
        case 1:
            cout << "Enter deposit amount: $";
            cin >> amount;
            myAccount.deposit(amount);
            break;
        case 2:
            cout << "Enter withdrawal amount: $";
            cin >> amount;
            myAccount.withdraw(amount);
            break;
        case 3:
            myAccount.displayBalance();
            break;
        case 4:
            cout << "Thank you for banking with us. Goodbye!\n";
            break;
        default:
            cout << "Invalid choice. Try again!\n";
    }
} while (choice != 4);

```

```
    return 0;  
}
```

Tips for a Great Presentation Rating

- **Highlight the Safety Check:** When explaining the withdraw function, emphasize how the code checks if `amount > balance`. Explain that this prevents the bank from losing money, showing you think about business logic while coding.
- **Keep it Simple:** If teachers ask how to improve it, tell them: *"In the future, I could add a file saving system (File I/O) to keep the data saved even after closing the program."* (This shows you know advanced concepts without having to code them right now).